



TaskPilot

The AI Operating Layer for the Browser

Technical Documentation & Implementation Guide

Version 1.0 · 2025 · taskpilot.cc

Stack	Next.js 14 · TypeScript · Supabase · Stripe · Upstash Redis
Extension	Plasmo · Manifest V3 · Shadow DOM · Service Worker
AI	OpenAI GPT-4.1-mini / GPT-4.1 · Semantic Caching · Token Optimizer
Hosting	Vercel · Supabase Edge · Upstash · PostHog Analytics

Table of Contents

01	Product Vision & Strategy	Positioning, target users, business model
02	System Architecture	Monorepo structure, tech stack, data flow
03	AI Orchestration Engine	Intent detection, tool routing, model selection
04	Smart Paste System	3-layer parsing, form detection, autofill
05	Token Optimization	Cost reduction strategies, semantic caching
06	Browser Extension	Manifest V3, content scripts, service worker
07	Database Schema	Supabase tables, RLS policies, views
08	API Design	Routes, auth, rate limiting, security
09	Stripe Integration	Plans, webhooks, billing portal
10	Security Architecture	CSP, rate limiting, abuse prevention
11	Analytics & Metrics	Usage tracking, cost monitoring, dashboards
12	Deployment Guide	Vercel, Supabase, Chrome Web Store

CHAPTER 01

Product Vision & Strategy

Positioning · Target Users · Business Model

1.1 Vision

TaskPilot is not another AI chatbot. It is the AI operating layer for the browser — turning every webpage into an intelligent workspace where users can talk to content, automate workflows, extract data, and delegate browser tasks to AI. The core philosophy is: Ask → Execute → Done.

1.2 Primary Positioning

Tagline	"Talk to any webpage instantly."
Category	AI Browser Extension / Productivity SaaS
Competitor feel	Cursor for the browser · Arc Browser AI · ChatGPT with execution
Differentiator	LLM plans, browser engine executes — controlled, reliable automation

1.3 Target Users

- Recruiters (LinkedIn sourcing, ATS autofill)
- Sales teams (CRM data entry, lead capture)
- E-commerce operators (product data extraction)
- Agencies (client reporting, data collection)
- Founders (competitive research, lead gen)
- Virtual assistants (browser task automation)
- Researchers (structured data extraction)
- Growth marketers (SEO audits, SERP analysis)

1.4 Business Model

Plan	Price	AI Actions/mo	Key Features
Free	\$0	30 actions 5 exports	Smart Paste · Basic sidebar · CSV export
Pro	\$19/mo \$190/yr	Unlimited	All features · CRM integrations · Browser actions Premium AI · Workflow history · Priority support

Enterprise	Custom	Unlimited	SSO · Team management · API access Dedicated support · Custom models
------------	--------	-----------	---

CHAPTER 02

System Architecture

Monorepo · Data Flow · Infrastructure

2.1 Monorepo Structure

TaskPilot uses a Turborepo monorepo with four packages and two apps. All packages are TypeScript-first with strict mode enabled.

```
taskpilot/
  apps/
    web/           → Next.js 14 (landing page + dashboard + API)
    extension/     → Chrome Extension via Plasmio (Manifest V3)
  packages/
    ai-engine/     → Orchestrator, optimizer, semantic cache, tool registry
    browser-tools/ → Smart paste, form detection, export utilities
    shared/        → Types, constants, PLAN_LIMITS, utilities
    ui/            → Design system: buttons, cards, inputs
  supabase/
    migrations/    → PostgreSQL schema (001_initial_schema.sql)
    functions/     → Edge Functions (ai-proxy)
  scripts/
    package-extension.js → Chrome Web Store packager
```

2.2 Request Data Flow

Step	Component	Description
1	Chrome Extension	User triggers action (Smart Paste / Sidebar / Command)
2	Content Script	Extracts page context, forms, selected text via Shadow DOM
3	Background SW	Rate limits request, appends session token, routes to API
4	Next.js API	Auth check, security validation, cache lookup
5	Semantic Cache	Returns cached result if hit (34%+ cache rate)
6	AI Engine	Heuristic routing → token optimization → OpenAI call
7	Supabase	Log request, increment usage counter, update metrics
8	Response	Result streamed back to extension → rendered in sidebar

2.3 Infrastructure

Service	Provider	Purpose
Web App + API	Vercel (Edge + Node)	Landing page, dashboard, API routes
Database	Supabase (PostgreSQL 15)	User data, usage logs, workflows
Edge Functions	Supabase Edge	AI proxy with rate limiting
Cache + Rate Limits	Upstash Redis	Semantic cache, IP throttling
AI Models	OpenAI	GPT-4.1-mini (default), GPT-4.1 (complex)
Payments	Stripe	Subscriptions, webhooks, portal
Analytics	PostHog	Usage tracking, funnels, retention
CDN	Vercel Edge Network	Global delivery, 30+ PoPs

CHAPTER 03

AI Orchestration Engine

Intent Detection · Tool Routing · Model Selection

3.1 Overview

The AI engine in packages/ai-engine/ is the brain of TaskPilot. It receives user intent, selects tools, optimizes tokens, routes to the appropriate model, and assembles the final result. The design principle: LLM = Planner, Browser Engine = Executor.

3.2 Execution Pipeline

Step	Description
1. Cache Check	SemanticCache.get() — SHA-hash of task+content+url. 34% avg hit rate.
2. Heuristic Routing	For regex-friendly tasks (email, price extraction), skip AI entirely. ~40% of requests.
3. Intent Detection	Map task type to TASK_TOOL_MAP → select execution tools.
4. Token Optimization	TokenOptimizer strips boilerplate, compresses whitespace, limits content per task type.
5. Model Selection	gpt-4.1-mini for simple/moderate. gpt-4.1 for complex tasks (replies, meeting notes).
6. AI Call	OpenAI structured JSON output. System prompt is minimal — task-specific, under 100 tokens.
7. Result Assembly	Parse AI output, merge with heuristic results, validate.
8. Cache Write	Store result with TTL (1h default, 24h for stable content like translations).

3.3 Model Routing Logic

```
gpt-4.1-mini → summarize, translate, extract_emails, extract_prices,
              smart_paste, rewrite, extract_tables
gpt-4.1      → generate_reply, meeting_notes, custom (complex user prompts),
              browser_action_plan
```

3.4 Tool Registry

20 tools registered in packages/ai-engine/src/tools/tool-registry.ts. Each tool has a phase (1=local, 2=AI, 3=integration), cost weight (0.05–0.6), and handler function. Phase 1 tools run without any AI cost.

Tool	Phase	Cost Weight	Description
extract_visible_content	1	0.10	Raw page text extraction

extract_emails	1	0.05	Regex email extraction (no AI)
extract_prices	1	0.05	Regex price/currency extraction
extract_links	1	0.05	URL extraction from content
extract_tables	1	0.10	Table data from DOM
detect_forms	1	0.10	Form field detection + labeling
summarize_content	2	0.50	AI-powered page summary
rewrite_text	2	0.40	Tone-specific text rewriting
translate_content	2	0.60	Multi-language translation
generate_reply	2	0.50	Context-aware reply generation
autofill_fields	2	0.30	Intelligent form field filling
export_csv	2	0.10	CSV export from extracted data
export_excel	2	0.10	XLSX export with formatting
export_pdf	2	0.20	PDF export from page content
push_to_hubspot	3	0.20	HubSpot CRM integration
push_to_salesforce	3	0.20	Salesforce CRM integration
create_notion_page	3	0.20	Notion database entry creation

CHAPTER 04

Smart Paste System

3-Layer Parsing · Form Detection · Animated Autofill

4.1 User Experience

The Smart Paste system is the viral MVP feature. The user copies messy text (e.g. a LinkedIn profile or business card) and clicks Smart Paste (or presses Alt+V). TaskPilot detects form fields on the page, maps clipboard data to fields intelligently, and autofills them with an animated highlight effect.

4.2 Three-Layer Parsing

Layer 1: Regex (no AI cost)

- Email: `/[a-zA-Z0-9._%+\-]+\@[a-zA-Z0-9.\-]+\.[a-zA-Z]{2,}/`
- Phone: international format with country codes
- URLs, LinkedIn/Twitter handles, zip codes
- Dates in multiple formats

Layer 2: Heuristics (no AI cost)

- "Title at Company" pattern → splits jobTitle + company
- Title-case 2+ words → firstName + lastName
- JOB_TITLE_SIGNALS array (CEO, CTO, VP, Director, ...)
- COMPANY_SIGNALS (Inc, Ltd, Corp, Group, ...)
- DOM proximity labels (for/aria-label/sibling/ancestor)
- Confidence scoring: 0.0–1.0

Layer 3: AI (only if confidence < 0.7)

- gpt-4.1-mini with structured JSON output
- System prompt under 80 tokens
- Only called for ambiguous/complex clipboard content
- Merges with heuristic results
- Skipped for >60% of requests

4.3 Form Detection Engine

The content script's `detectFormFields()` function finds all visible form elements and extracts semantic labels via 4 methods (in priority order):

Priority	Method	Example
1	for= attribute on label	<code><label for="email">Email</label></code>

2	aria-label on input	<code><input aria-label="Email address"></code>
3	Ancestor <code><label></code> element	<code><label>Email <input type="email"></label></code>
4	Preceding sibling text	<code>Email <input type="email"></code>

4.4 React/Vue Compatibility

Standard DOM value assignment breaks React/Vue controlled inputs. The `animatedAutofill()` function uses the native input value setter + dispatches `InputEvent` with `bubbles:true` to ensure framework state syncs correctly. This is critical for HubSpot, Salesforce, and Airtable.

```
Object.getOwnPropertyDescriptor(HTMLInputElement.prototype, 'value')
  .set.call(input, value)
input.dispatchEvent(new Event('input', { bubbles: true }))
input.dispatchEvent(new Event('change', { bubbles: true }))
```

CHAPTER 05

Token Optimization

Cost Reduction · Semantic Cache · Analytics

5.1 Strategy Overview

AI inference cost is the primary variable cost at scale. TaskPilot's target is <\$0.00015 per request after all optimizations. The five-layer approach achieves 60-80% cost reduction versus a naive implementation.

Local-first parsing

Heuristics handle ~40% of requests, regex handles ~20%. No AI call needed.

Semantic caching

34%+ cache hit rate. Same page + same task = instant result at \$0 cost.

Token limits per task

smart_paste: 500 chars · summarize: 3000 · export: 4000 max content.

Boilerplate stripping

Removes cookie notices, nav menus, footer links (~20-30% reduction).

Model routing

gpt-4.1-mini by default. 10x cheaper than gpt-4.1 for suitable tasks.

5.2 Per-Task Content Limits

Task	Max Content (chars)	Rationale
smart_paste	500	Only clipboard text — short by nature
extract_emails	10,000	Needs full page scan
extract_prices	8,000	Product listings can be long
summarize	3,000	First 3k chars captures the gist
translate	4,000	Balance quality vs cost
generate_reply	2,000	Message context is usually short
export_pdf	4,000	Section limit for AI-assisted export
custom (user prompt)	6,000	Broad queries may need more context

5.3 Cost Model

```
gpt-4.1-mini: $0.00000015 / token (input + output)
gpt-4.1:      $0.0000025 / token
```

Average request: ~800 tokens after optimization

Cost per request (mini, no cache): \$0.00012

Cost per request (after 34% cache): \$0.000079

Pro plan break-even at \$19/mo: ~240,000 requests

CHAPTER 06

Browser Extension

Manifest V3 · Content Scripts · Service Worker

6.1 Architecture

The Chrome extension uses Plasmo Framework with Manifest V3. Three core components: background service worker (persistent state, rate limiting, context menus), content script (page interaction, Shadow DOM sidebar, form detection), and popup UI (quick actions, AI input, recent history).

6.2 Permissions

Permission	Type	Purpose
storage	Required	Session token, usage counters, settings
clipboardRead	Required	Smart Paste — read clipboard content
clipboardWrite	Required	Copy AI results to clipboard
activeTab	Required	Access current tab content and URL
scripting	Required	Execute content scripts dynamically
contextMenus	Required	Right-click menu (Summarize, Extract, etc.)
alarms	Required	Hourly session sync for logged-in users
tabs	Optional	Access tab list for multi-tab workflows
<all_urls>	Optional	Full URL access for advanced automation

6.3 Shadow DOM Isolation

The sidebar is injected into a Shadow DOM container, completely isolating TaskPilot's styles from the host page. This prevents CSS conflicts on 99.9% of websites and ensures consistent rendering across all domains.

```
const host = document.createElement('div')
host.id = 'taskpilot-root'
const shadow = host.attachShadow({ mode: 'closed' })
const iframe = document.createElement('iframe')
iframe.src = chrome.runtime.getURL('sidebar.html')
shadow.appendChild(iframe)
```

6.4 Keyboard Shortcuts

Shortcut	Action
Alt+T	Open TaskPilot (popup)
Alt+V	Smart Paste (autofill from clipboard)
Alt+S	Toggle AI Sidebar
Alt+K	Open AI Command Palette
Escape	Close sidebar / dismiss notification

CHAPTER 07

Database Schema

Tables

- profiles — user accounts, plan, stripe_customer_id
- anonymous_sessions — fingerprint + IP hash, 24h expiry, usage counters
- subscriptions — Stripe subscription state (syncd via webhook)
- ai_requests — all AI calls: task, model, tokens, cost, cached flag
- usage_periods — monthly counters per user (resets at month start)
- productivity_metrics — daily actions, forms filled, time saved per user
- workflows — saved automation sequences
- response_cache — semantic cache warm data
- saved_prompts — 6 defaults seeded + user custom prompts

Security

- Row Level Security (RLS) enabled on ALL user tables
- Policy: users can only SELECT/UPDATE their own rows
- Service role key only used server-side (never in browser)
- Anonymous sessions use SHA-256 fingerprint (IP + UA + timezone)

Views

- v_daily_stats — DAU, token costs, cache hits, task breakdown
- v_user_ltv — lifetime value per user (total cost vs revenue)

CHAPTER 08

API Design

Routes

- POST /api/ai/smart-paste — clipboard parsing + form field mapping
- POST /api/ai/process — all sidebar tasks (summarize, extract, etc.)
- POST /api/stripe/webhook — Stripe events (signed, idempotent)
- GET/POST/DELETE /api/auth/session — session management
- POST /api/export — CSV / Excel / JSON generation

Auth Pattern

- Supabase Auth (JWT) for authenticated users
- Anonymous session token in X-Session-Token header for guests
- Extension sends Origin: chrome-extension://<id>
- CORS whitelist: taskpilot.cc + localhost:3000 + chrome-extension://*

Rate Limits (per IP per minute)

- smart-paste: 30 req/min
- process: 60 req/min
- auth: 10 req/min
- export: 20 req/min

CHAPTER 09

Stripe Integration

Plans

- Free: \$0 — enforced via Supabase usage counters (no Stripe needed)
- Pro Monthly: \$19/mo (STRIPE_PRICE_PRO_MONTHLY)
- Pro Annual: \$190/yr (STRIPE_PRICE_PRO_ANNUAL) — 17% discount
- Enterprise: custom pricing via Stripe quote

Webhooks

- checkout.session.completed — create subscription record
- customer.subscription.updated — sync plan changes
- customer.subscription.deleted — downgrade to free
- invoice.payment_succeeded — log billing event
- invoice.payment_failed — flag account, email user

Trial

- 7-day free trial on Pro (no card required for Free)
- Trial via Stripe checkout session trial_period_days: 7
- Promotion codes enabled (allow_promotion_codes: true)

CHAPTER 10

Security Architecture

Client-Side

- Shadow DOM isolation — no CSS/JS conflicts with host pages
- Minimal permissions — activeTab only, no broad host permissions required
- Content Security Policy hardened in manifest.json
- All secrets server-side only — extension uses API proxy

Server-Side

- Rate limiting: Redis sliding window per IP (INCR + EXPIRE)
- Burst detection: >15 requests in 5 seconds = temporary block
- Input sanitization: strip <script>, javascript:, on* handlers
- Content length limits: 20k chars max per request
- Extension CORS: chrome-extension:// origin whitelist
- Strict CSP headers on all responses
- X-Frame-Options: DENY, HSTS, Permissions-Policy

AI Safety

- task_type whitelist — only 20 known task types accepted
- URL validation — no internal/localhost URLs
- Content sanitization before AI call
- Structured outputs only — JSON schema enforced

CHAPTER 11

Analytics & Metrics

User Metrics

- Daily Active Users (DAU), Weekly Active Users (WAU)
- Feature adoption rates (Smart Paste vs Sidebar vs Actions)
- Conversion: free → pro
- Retention D1/D7/D30
- Session quality (actions per session, completion rate)

AI Cost Metrics

- Tokens used per user, per task type
- Cache hit rate (target: >30%)
- Average cost per request
- Model distribution (mini vs full)
- Cost per MAU estimate

Business Metrics

- MRR, ARR growth
- CAC (cost per acquired user)
- LTV per plan tier
- Churn rate (monthly)
- Expansion revenue (free → pro upgrades)

CHAPTER 12

Deployment Guide

Web App (Vercel)

- 1. Connect GitHub repo to Vercel
- 2. Set Root Directory: apps/web
- 3. Add all env vars from .env.example
- 4. Deploy: `vercel --prod`
- 5. Verify: `https://taskpilot.cc/api/auth/session`

Database (Supabase)

- 1. Create project at `supabase.com`
- 2. Run: `supabase db push --linked`
- 3. Enable Anonymous Sign-ins in Auth settings
- 4. Deploy edge function: `supabase functions deploy ai-proxy`
- 5. Set function secrets via Supabase dashboard

Chrome Extension

- 1. Build: `cd apps/extension && npm run build`
- 2. Package: `node scripts/package-extension.js`
- 3. Upload `dist/taskpilot-extension-prod-*.zip` to Chrome Web Store
- 4. Submit for review (5-7 business days)
- 5. Update `manifest.json` version for each release